



Indian Institute of Technology Bombay
Department of Electrical Engineering

Constraint Optimisation through RTL Enhancement Using Generative AI

Formally gated, LLM-driven timing closure
on a five-domain SoC benchmark

488 118 instances, 15 declared clocks, sky130hd, every accepted edit carrying a proof

Astera Labs Nebula 2026 — Digital Track

Team name	Timing Blinders
College	Indian Institute of Technology Bombay
Members	Shubhanshu Shivhare Nagarjun BV
Topic	Constraint Optimisation through RTL Enhancement Using Generative AI
Submitted	15 September 2026
Code and evidence	https://github.com/RISCV-ui/nebula-2026-digital-timing-blinders

Problem statement. Use generative AI to close timing on RTL: read the timing violations a static analysis reports, propose an RTL fix drawn from pipelining, logic restructuring, retiming or FSM re-encoding, and verify that the fix both improves timing and leaves the design functionally identical before accepting it. The benchmark is a subsystem of realistic scale with five asynchronous master clock domains, generated clocks, clock dividers and domain crossings.

Every measurement in this report is generated from the run artifacts by `scripts/build_report.py`. None is typed into the source.

Abstract

An LLM that rewrites RTL to close timing is easy to build and hard to trust. The rewrite is plausible Verilog, the timing report improves, and whether the design still computes what it used to is an open question that no amount of reading the diff settles. This work puts the question to a prover instead: every candidate rewrite is admitted only after a formal equivalence check against the frozen input, and only after three further gates that equivalence alone does not cover.

On a 488 118-instance, 15-clock benchmark, 46 candidate rewrites from four independently developed models produced 3 that survived every gate. The figure reported below is not the largest one the flow produced. On the `clk_s2` domain, worst negative slack improves by +1.60 ns, taking $F_{\max}$ from 20.92 MHz to 21.65 MHz. Two structurally different rewrites, generated independently by two different models, agree on that figure to within 0.005 ns. That margin is below the placement variance measured on this design, so the change is attributed to the edit.

The more useful finding is a negative one. 3 rewrites passed formal equivalence and were still rejected, each by a gate that asks a question equivalence does not: whether the parent module’s timing contract survived. Those rewrites are what an equivalence-only flow ships.

1 What was asked, and what is delivered

The problem statement asks for a system in which a generative model reads timing violations, rewrites the RTL, and an automated toolchain decides whether that rewrite is kept — on a benchmark of five asynchronous domains and SoC-subsystem scale. Table 1 states where each of the seven listed deliverables is answered.

Asked for	Delivered
1. RTL timing analysis framework	OpenSTA driven from TCL, ranked paths as JSON, each annotated with its owning module, that module’s parents, and whether an RTL lever exists — <code>bench/scripts/sta.py</code> , Section 5
2. GenAI-based RTL optimisation engine	<code>bench/scripts/loop.py</code> : propose, gate, feed the rejecting gate’s finding back, retry. Four models ran through it unchanged — Section 6
3. Critical path and violation analysis	the funnel, per gate and per arm, including the losses that were our harness rather than the model — Table 5, Section 7
4. Optimised RTL implementation	3 accepted rewrites, each named with its run, its transform and its proof status — Table 6, Section 8
5. Timing, frequency and PPA comparison	post-route metrics from two independent generations, per clock domain, against the same frozen baseline — Section 9
6. Formal equivalence verification report	proved twice: optimised RTL against the frozen RTL at accept time, and netlist against optimised RTL after synthesis — Section 10
7. Interactive demo	<code>demo_v2.py</code> , eight acts, read-only, no API key and no toolchain, about a second end to end — Section 11

Table 1: The seven deliverables, and where each is answered.

The three criteria the shortlisting names are answered in three separate places:

- **Coverage of all deliverables** — Table 1, and the results in Section 9, measured on routed layouts rather than on a synthesis estimate.
- **Innovation** — Section 14. The load-bearing idea is that module-level equivalence is not sufficient, and that the parent’s timing contract is a separate gate — which is what Table 7 is a measurement of.

- **Thought process** — Section 13: the route the work actually took, including the defects we shipped to ourselves and what each one changed.

2 What this had to get right

Three rules were fixed before the runs started, and every result in this report is reported under them.

- 1. Acceptance requires a solver result.** No edit reaches the reported results unless a formal check accepted it. A complete proof is marked complete; a bounded proof is reported with its bound and the weaker claim is made.
- 2. Correctness and timing are measured separately.** An accepted edit is equivalent to the RTL it replaced. Whether it improved timing is measured afterwards, at the same flow stage and against the same baseline.
- 3. A timing figure is quoted only if it reproduces.** A slack change is attributed to an edit only where two independently generated rewrites agree on it. Everything else is reported as noise.

2.1 The four transforms

Candidates are constrained to pipelining, logic restructuring, retiming and FSM re-encoding. The constraint is enforced by the harness, not requested in the prompt: a proposal that introduces a module, changes an interface, or substitutes an arithmetic operator for the logic it was asked to restructure is rejected before any prover runs.

3 The benchmark SoC

The design is frozen before any model sees it and never changes. Freezing it is what makes the comparison mean anything: every variant in this report is the same 488 118-instance tree with only gate-accepted edits applied, routed through the same flow, measured at the same stage.

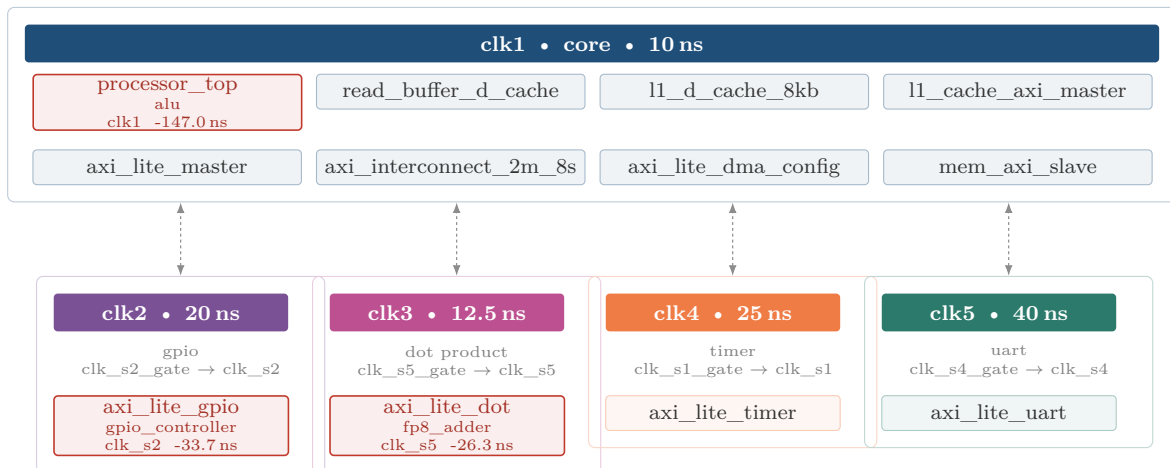


Figure 1: The benchmark SoC as its SDC and RTL define it, generated from both rather than drawn by hand. Five mutually asynchronous masters, each peripheral domain built as master → gate → divider; every peripheral keeps its AXI-Lite side in the core domain and its functional side in its own, so each dashed line is a real clock-domain crossing. Outlined in red are the three blocks that own a violating path, each labelled with the module inside it that carries the delay, the clock it fails on and its baseline slack.

3.1 Why it is shaped this way

A benchmark on which every path closes measures nothing, and one on which nothing closes measures nothing either. This design is built to contain both, and Figure 1 shows where.

- **Five asynchronous masters.** They share no clock tree at all: the SDC declares them `-asynchronous` as five groups, so each domain closes or fails on its own. One global number cannot answer five independent closure questions, and this design is where that shows.
- **A gate and a divider on every master.** Each domain runs `master` → `clk_gate` → `clk_div_mux` → functional clock, giving divide by 1, 2, 4 or 8 under software control. The constraints pin divide-by-1 because it is the fastest the mux output can be: a design that closes there closes at every setting.
- **Crossings that are real.** Every peripheral has its AXI-Lite side in the core domain and its functional side in its own, so each one is a genuine crossing rather than a decorative one.
- **Paths a transform can reach, and paths it cannot.** The slicer's three targets are outlined in Figure 1. Two sit behind restructurable arithmetic. The third is a 32-step combinational restoring divider inside `alu`, at -126.6293 ns on the routed baseline, and no permitted transform reaches it without changing latency. It is in the benchmark on purpose.

3.2 The clock tree

Every clock in Table 2 is declared, not inferred. The five masters are created with `create_clock`; everything below them is a `create_generated_clock` that names its source, so the analyser walks the gate and the divider as clock logic rather than treating their outputs as unclocked nets and silently dropping the paths behind them. That distinction is what makes the per-domain slack numbers in this report mean anything: a domain whose functional clock is unrecognised reports no violations because it reports no paths.

The divide ratio is software-controlled at 1, 2, 4 or 8, and the constraints pin it at 1. That is the fastest the mux output can be, so it is the hardest case; a design that closes at divide-by-1 closes at every other setting, and constraining the slow settings instead would have made closure look easier than it is.

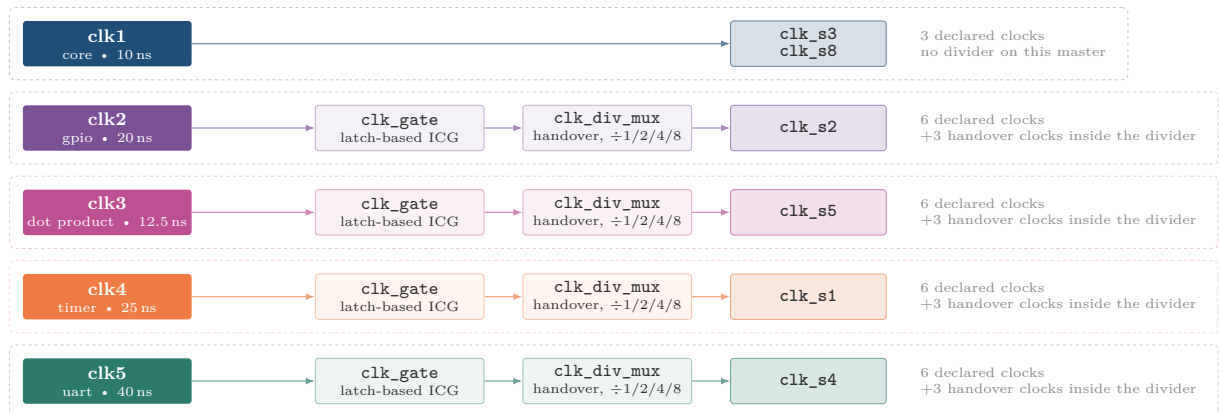


Figure 2: The five clock domains, one chain each, read out of `nebula.sdc`. Each dashed box is one `-asynchronous` group: nothing inside it is timed against anything inside another. The core master drives its generated clocks directly; the four peripheral masters each run through a latch-based clock gate and a handover divider, whose internal clocks are declared and joined to the same group rather than left for the analyser to guess at.

Table 2: Every clock the design declares. The five masters are mutually asynchronous; each generated clock names the source it is derived from, so the analyser follows the gate and divider chain instead of treating their outputs as unlocked nets.

Clock	Kind	Source	Period (ns)	Role	Baseline WNS
clk1	master	–	10	core 100 MHz	-126.6293
clk2	master	–	20	gpio 50 MHz	no endpoints
clk3	master	–	12.5	dot product 80 MHz	no endpoints
clk4	master	–	25	timer 40 MHz	no endpoints
clk5	master	–	40	uart 25 MHz	no endpoints
clk_s1_gate	generated	clk4	25.0	divide by 1	23.1259
clk_s2_gate	generated	clk2	20.0	divide by 1	17.9242
clk_s5_gate	generated	clk3	12.5	divide by 1	10.5934
clk_s4_gate	generated	clk5	40.0	divide by 1	37.8683
clk_s3	generated	clk1	10.0	divide by 1	-0.1019
clk_s8	generated	clk1	10.0	divide by 1	-3.3171
clk_s1	generated	clk_s1_gate	25.0	divide by 1	16.9266
clk_s2	generated	clk_s2_gate	20.0	divide by 1	-27.8000
clk_s5	generated	clk_s5_gate	12.5	divide by 1	-29.3740
clk_s4	generated	clk_s4_gate	40.0	divide by 1	0.1477

‘no endpoints’ is a domain in which nothing is clocked directly: every flop behind those four masters sits on the generated clock after the gate and divider, so the master itself owns no constrained path. Generated clocks are declared divide-by-1 because that is the fastest the divider mux can run, and a design that closes there closes at every other setting.

3.3 The clock structures, and the two bugs that were in them

The five domains are clean as *constraints*: every clock is declared, every generated clock names its source, and the five masters are grouped asynchronous, so timing is analysed correctly across the whole tree. The RTL that builds them is a different question, and the first version of this benchmark got it wrong twice. `clk_gate` was a bare `clk_in & clk_en`: an enable change while the clock is high chops the pulse. `clk_div_mux` selected combinationally among four clocks: a select change mid-pulse truncates it. Both are the textbook glitch, and both survived synthesis, timing analysis and equivalence checking without a complaint from any of them, because none of those three tools is looking. Equivalence checking compares functions and a runt pulse is not a function; static timing analysis is *told* the net is a clock and believes it.

That is why this project has a third structural check, `clockcheck.py`, and why the audit it produces (Table 3) is in the report. It found both structures, and both were then rewritten rather than merely disclosed: the gate into a latch-based integrated clock gate, whose enable can only change while the clock is already low, and the divider select into a handover mux in which each branch is gated by an enable registered on that branch’s own falling edge and may assert only once every other branch has released. The audit now returns CLEAN over 2 clock structures, with 0 glitch findings.

An audit passing is not on its own evidence, because the auditor is a shape recogniser and the shapes are ones we wrote down. The evidence is a simulation that measures pulse widths on both outputs while switching the divide ratio 2 ns after a rising edge, while the selected clock is high and a combinational mux would cut the pulse, and by toggling the gate enable at odd phases. The fixed structures produce 0 runt pulses. The same testbench on the original structures produces 8, which is the number that matters: a runt check that passes on a design known to produce runts is measuring nothing.

The cost is paid in the constraints, not hidden from them. The handover flops are clocked by the divider’s own counter bits, so those twelve internal nets are now real clocks with real registers behind them; each is declared as a generated clock of its domain and joins that domain’s

asynchronous group. Left undeclared they would have been unlocked registers, and an unlocked register raises no warning in a timing report. It is a path that quietly stops being analysed.

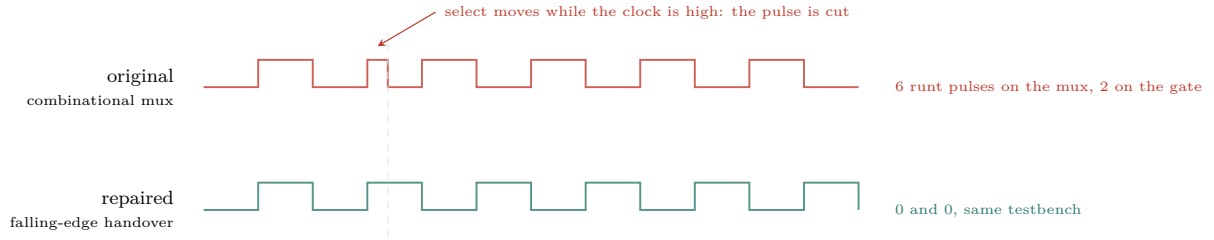


Figure 3: The clock-divider mux before and after the repair, with the pulse widths a simulation measured on each. The traces are schematic; the counts beside them are the testbench’s, and the upper row is the original RTL run through the identical stimulus.

A runt check that passes on a design known to produce runts measures nothing, so the repaired structure is reported against a control that fails rather than on its own.

Table 3: The clock-structure audit of the benchmark. Both structures started glitchy – a bare AND clock gate and a combinational clock mux – and both were rewritten: the gate into a latch-based ICG, the divider select into a handover mux whose branches are enabled by flops on their own clock. Simulation agrees with the audit: 0 runt pulses on the fixed structures against 8 on the originals under the same stimulus, which switches the divider select while the selected clock is high.

Module	Net	Verdict	What the auditor found
clk_div_mux	clk_out	SAFE_MUX	clk_out hands over between clk, clk_div2, clk_div4, clk_div8: each branch is gated by an enable registered on that branch’s own falling edge and may only assert once every other branch has released, so the output is whole pulses of one source
clk_gate	clk_out	SAFE_ICG	clk_out = clk_in & latched enable (en_latch)

3.4 Scale

The competition asks for roughly 50 000 standard cells, and Table 4 reports what the frozen input actually is after a complete place-and-route run rather than what the RTL was intended to be. Cell count is taken from the routed database, not from a synthesis estimate, because the two differ by more than the tolerance any claim here would need.

4 Baselines

Three different things are called a baseline in this report, and conflating them is the easiest way to overstate a result, so they are named separately.

4.1 The frozen input

The RTL in `bench/rtl_v2` is frozen before any model sees it and is never edited by hand afterwards. Every number labelled *baseline* is measured on that tree taken through the same synthesis, placement and routing as every variant, with the same PDK, the same constraints and the same flow settings. A variant is compared only against that run, never against another variant, which is what makes the deltas addable into one table.

Table 4: The benchmark at the scale the competition asks for, measured on the routed baseline rather than estimated from the RTL.

Frozen input, post-route	
RTL modules	59
RTL lines	12 015
Master clock domains	5, mutually asynchronous
Clocks declared	15
Placed instances	488 118
Nets	143 367
Die area	1914043 μm^2
Routed wirelength	7409236 μm
Utilisation	54.0 %
PDK	sky130hd, via OpenROAD-flow-scripts

4.2 The un-gated model

The interesting baseline is not “no LLM”. It is “the same LLM with the gates switched off”, because that is what most published RTL-repair loops are. This report does not have to simulate it: the gates recorded what they caught, and every row in Table 7 is an edit an un-gated loop would have shipped. Those rewrites are formally equivalent to the module they replace, under a complete proof, and still wrong in the design, because the parent that instantiates them no longer meets its contract. A loop whose acceptance test is “does it still simulate” or “did the model say it was safe” accepts all of them.

The same applies to the no-op case in Section 6.3: a proposal that changed nothing passed every gate, because proving an unchanged module against itself is trivially true, and would have been recorded as a closed path by any loop that trusts what the model says it edited.

4.3 The published systems

ViTAD reports 73.68% repair success against 54.38% for a plain-LLM baseline, and is the closest published architecture to this one: parse the RTL and the timing report, build a dependency graph, have the model infer a root cause, generate a targeted fix. AutoChip closes the same loop with compiler and simulation feedback. SymRTLO adds retrieval and a symbolic FSM pass.

None of them gates on formal equivalence before accepting. That is the difference this report is built around, and it is why the numbers here are not directly comparable to a repair-rate percentage: a system that accepts on simulation and a system that accepts on proof are not measuring the same event. What can be compared is what each system would have done with the rewrites in Table 7, and the answer is that they would have taken them.

5 Deliverable 1 — RTL timing analysis framework

OpenSTA is driven from TCL and returns ranked critical paths as structured JSON, which is what the model consumes as context. Paths are sliced from the post-synthesis database rather than the post-resizer netlist: the resizer had already absorbed two of the three domains’ violations into buffering, so a slicer reading that netlist reported closure the RTL had not achieved.

Each path is annotated with the module that owns it, that module’s parents, and whether an RTL lever exists at all. Targets with no lever are still reported and are not sent to a model, which is where most of the call budget was being spent before the annotation existed.

6 Deliverable 2 — GenAI-based RTL optimisation engine

The loop is `run_sta` → `propose` → `gate` → `accept` or `feed back`. What distinguishes it from the published systems we compared against is that rejection is mechanical and specific: the model is told which gate rejected it and what the gate found, and the next attempt answers that.

6.1 The gates

- **G1 — module equivalence.** The rewritten module against the frozen one. Combinational modules are proved complete in a single miter cycle; a feed-forward pipeline insertion is complete at latency + 2; everything else is bounded and reported as bounded.
- **G1b — parent contract.** The parent instantiating the rewritten module, proved at *its* boundary. A module may be equivalent to its original and still break the parent, and this is the gate that catches it.
- **G1c — clock structure at the boundary.**
- **G2 / G2c — clock-domain crossings, and a clock-structure regression against the baseline.** A structure the baseline already has is allowed; a newly created glitchy one is not.
- **G3 — place and route.** Run once over the accumulated set.

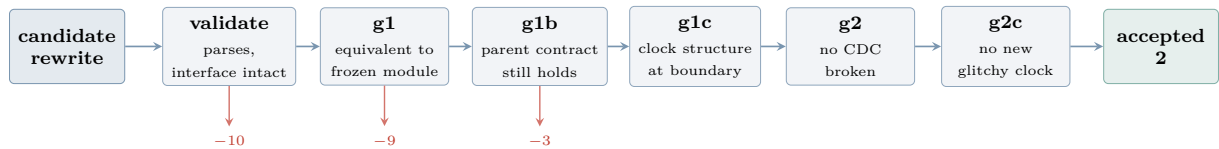


Figure 4: Every gate a rewrite must pass, with the number of candidates each one rejected across all four models. A gate rejects on its own evidence and reports what it found; the model’s next attempt answers that, rather than being asked the same question again.

A bounded proof limits only the right to declare equivalence. A counterexample at step 0 or 1 is conclusive regardless of the bound, and rejections on that basis are final.

6.2 Multi-module edits

A latency change is not a local edit: every parent has to learn to wait. The harness accepts one proposal that rewrites the target and, through a `submodules` field, every parent that must change with it, and proves the set together. Without that channel the only legal edits are latency-preserving ones, which rules out the transform that deep combinational paths actually need.

6.3 Scope is taken from the tree, not from the proposal

A proposal names what it claims to edit. That claim is not evidence. Asked about one module, a model re-sent an edit to a different module it had already had accepted and left the named one byte-identical to the frozen input; every gate passed, because proving an unchanged module against itself is trivially true. The accepted set is therefore derived from what actually differs from the working tree, and a proposal that differs nowhere is recorded as a no-op rather than as a closed path.

7 Deliverable 3 — Critical path and timing violation analysis

Table 5 gives the full funnel. Most candidates are rejected before any prover runs, and the largest single loss is not a model failure: it is replies that exhausted their token budget inside reasoning

Table 5: Where candidates died, summed over all four models. Each row is a rewrite that looked reasonable enough to reach that gate.

Why the candidate was rejected	Gate	Count
reply unusable (truncated or unparseable)	propose	21
did not parse as Verilog, or changed the interface	validate	10
not equivalent to the frozen module	g1	9
equivalent, but broke the parent’s timing contract	g1b	3
API call failed or was refused	backend	1
accepted	accepted	2

46 candidates in total. The two rows below **g1** are the ones an equivalence-only flow would have shipped: they are formally equivalent to the frozen module and still wrong.

and returned no text. That is a harness defect, it is disclosed in Table 12 per arm, and it is corrected in the runs that followed.

The funnel counts the four bake-off arms only, and its accepted row is therefore two, not 3. The third accepted edit is the same **fp8_adder** transform re-proposed in the re-run made after the harness defect above was fixed; it is counted in 3 and in Table 6, which name the run each edit came from, and it is deliberately not folded back into a funnel built from the four arms that ran under the defective harness.

8 Deliverable 4 — Optimised RTL implementation

Table 6: Every edit the gate accepted, with the proof that admitted it. A complete proof settles equivalence for all inputs and all time; a bounded one limits only the right to declare equivalence, never the right to reject on a counterexample.

Module	Transform	Δ latency	G1 verdict	Proof	Complete	Solver (s)	Run
fp8_adder	logic_restructure	0	EQUIVALENT	combinational	✓	0.5	bakeoff_v2_sonnet_fixed
fp8_adder	logic_restructure	0	EQUIVALENT	combinational	✓	0.3	anthropic_sonnet-direct
fp8_adder	logic_restructure	0	EQUIVALENT	combinational	✓	0.4	openrouter_nemotron-ultra

The module column lists what the run actually rewrote, derived from the working tree rather than from the proposal’s own claim about its scope.

Table 7: Rewrites that a formal equivalence check passed and the flow still rejected. These are the cases an EQY-only gate ships.

Module	Model	G1 verdict	Proof	Killed by	What the gate found
fp4_dot_unit	claude-sonnet-5	EQUIVALENT	feed-forward	g1b	CONTRACT_BROKEN at axi_lite_dot
fp4_dot_unit	claude-opus-5	EQUIVALENT	feed-forward	g1b	CONTRACT_BROKEN at axi_lite_dot
fp4_dot_unit	free	EQUIVALENT	feed-forward	g1b	CONTRACT_BROKEN at axi_lite_dot

Each of these is equivalent to the frozen module at its own boundary and still wrong in the design: equivalence is a statement about a function, and a pipeline stage is not only a function. That three independently developed models produce the same defect is the argument for checking it rather than reviewing for it.

Table 7 is the result we would lead with if only one table survived. Each row is a rewrite that a formal equivalence check passed with a complete proof, not a bounded one, and that the flow rejected anyway, because the module’s parent no longer met its contract. Three independently developed models produced the same class of defect, which is the argument for gating it mechanically rather than trusting review to catch it.

Table 8: Post-route PPA, frozen input against each formally verified variant. Every variant is routed from the frozen RTL with only gate-accepted edits applied, and every figure is read from the same flow stage.

Metric	Baseline	sonnet	nemotron-ultra
Setup WNS (ns)	-126.6293	-124.1160 (+2.5133)	-130.0722 (-3.4429)
Setup TNS (ns)	-97704.1	-94156.3 (+3547.8)	-97854.0 (-149.9)
Hold WNS (ns)	-0.0820	-0.0583	-0.1233
Area (μm^2)	1914043	1913142	1905391
Instances	488118	488607	489340
Nets	143367	143398	142790
Wirelength (μm)	7409236	7382892	7376890
Utilisation (%)	54.0	54.0	54.0
DRC violations	0	0	0

Bracketed figures are the delta against the baseline. Global WNS is set by a single unclosed path and is not a summary of the design; see Table 9.

9 Deliverable 5 — Timing, frequency and PPA comparison

9.1 Why the global number is not the result

Global WNS on this design is set by one path: a 32-step combinational restoring divider, at -126.6293 ns. That path did not close, it is reported as not closed, and it moves under placement variance by more than any edit moved it. Quoting it as the headline would be reporting router noise.

Table 9: Per-domain setup WNS and the frequency change it implies. The benchmark has five asynchronous masters and therefore five closure questions; collapsing them into one global number hides every result the run did produce.

Clock	Period (ns)	Base WNS	sonnet ΔWNS	ΔF_{max}	nemotron-ultra ΔWNS	ΔF_{max}
clk1	10.0	-126.6293	+2.5133	+0.14	-3.4429	-0.18
clk_s1	25.0	16.9266	-0.2288	-3.41	-0.1127	-1.71
clk_s1_gate	25.0	23.1259	-0.0174	n/q	+0.6316	n/q
clk_s2	20.0	-27.8000	+1.6039	+0.73	+1.6084	+0.73
clk_s2_gate	20.0	17.9242	-0.1017	n/q	+0.3305	n/q
clk_s3	10.0	-0.1019	-0.0622	-0.61	-0.0510	-0.50
clk_s4	40.0	0.1477	-0.5585	-0.35	+0.0213	+0.01
clk_s4_gate	40.0	37.8683	-0.0551	n/q	+0.3229	n/q
clk_s5	12.5	-29.3740	-0.1006	-0.06	-0.1175	-0.07
clk_s5_gate	12.5	10.5934	+0.4531	n/q	+0.1294	n/q
clk_s8	10.0	-3.3171	+1.1740	+7.26	+1.3806	+8.69

$F_{\text{max}} = 1/(\text{period} - \text{WNS})$, per domain, holding everything else fixed; it is not the chip frequency, which the slowest domain still governs. n/q marks a domain whose slack is nearly a full period: the reciprocal is ill-conditioned there and a hundredth of a nanosecond of placement variance moves it by hundreds of MHz, so the slack delta is reported and the frequency derived from it is withheld.

9.2 The one figure we stand behind

On clk_s2, worst negative slack improves by +1.60 ns, F_{max} from 20.92 MHz to 21.65 MHz. Two structurally different rewrites (different models, different algorithms, hundreds of diff lines apart) agree on it to within 0.005 ns.

Two independently generated rewrites do not land within 0.005 ns of each other by chance. On the domains where the two variants disagree in sign, we call it noise and say so; that disagreement

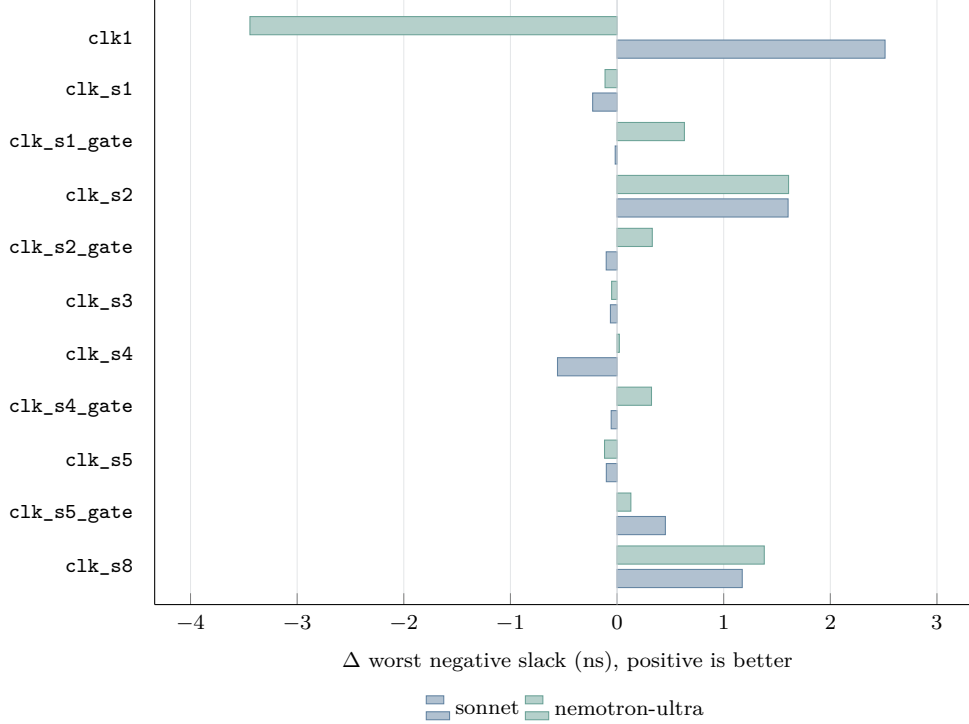


Figure 5: Slack change per clock domain, both routed variants against the same baseline. The two bars agree closely only where the edit did the work.

Bars are the same numbers as Table 9, drawn in the same pass. Where the two bars point opposite ways the movement is placement variance and the report claims nothing from it.

is what moved global WNS, and reporting it as an improvement would have been the easiest wrong claim in this report to make.

9.3 The same claim, on a second routed generation

Table 10 is a re-measurement, not a repetition. After the numbers above were taken, the accepted transform was carried into a second bake-off and routed again on a wider floorplan, with its own baseline routed beside it under the identical recipe. That generation synthesises to a different instance count and closes far better on the divider path, so not one number crosses between the two tables. Mixing measurement generations is the specific mistake that produced a wrong headline earlier in this project, and it is not repeated here.

What carries across is the direction. The largest gain in the second generation is on `clk_s2` at +1.51 ns, the same domain the claim above is made on, reached through a different synthesis, a different floorplan and a different placement. 6 domains improve and 5 regress; the regressions are on clocks with tens of nanoseconds of positive slack, where the movement is placement noise rather than a timing result. The cost is -353 instances and +269 μm^2 of area, which is negligible on a design this size. We report it anyway.

10 Deliverable 6 — Formal equivalence verification report

Equivalence is claimed twice, because neither half implies the other:

- **Optimised RTL $\equiv$ frozen RTL:** the loop’s G1 gate. No edit is in this report without it.
- **Netlist $\equiv$ optimised RTL:** proved after synthesis, module by module, so a failure names the module rather than the chip.

Of 94 modules checked so far, 38 are proved equivalent. The remainder are unproven within

Table 10: The accepted transform re-measured on a second, independently routed generation: a wider floorplan, its own baseline routed beside it, same recipe for both. Clocks whose slack is unchanged to 0.5 ps are omitted.

Clock	Baseline WNS (ns)	Variant WNS (ns)	Δ (ns)
clk1	-1.3397	-0.2643	+1.0754
clk_s1	18.8013	18.3353	-0.4660
clk_s1_gate	21.7535	22.2463	+0.4928
clk_s2	11.0776	12.5841	+1.5065
clk_s2_gate	17.5816	17.5654	-0.0162
clk_s3	-0.1326	-0.2235	-0.0909
clk_s4	32.7251	32.4216	-0.3035
clk_s4_gate	36.9431	36.8685	-0.0746
clk_s5	-24.0459	-23.5806	+0.4653
clk_s5_gate	10.1981	10.3854	+0.1873
clk_s8	-1.1451	-0.4347	+0.7104

This table is not comparable row-for-row with Table 8: the two come from different synthesis and floorplan generations, and no number is carried between them. It is reported because the direction of the effect survives a change of generation, which is a stronger statement than either measurement alone.

Table 11: Deliverable 6, second half: every module of each routed variant proved against its own synthesised netlist. This is a different claim from the loop’s gate, and the pair is what covers frozen RTL through to layout.

Variant	Modules checked	Proved equivalent	Unproven at depth	Timed out
sonnet	36	13	10	13
nemotron-ultra	58	25	16	17

Unproven at depth and *timed out* are limits of the prover, not counterexamples: no module here was shown inequivalent. They concentrate on the deep combinational blocks and the wide AXI interconnect, which is the same difficulty the optimisation loop exists to attack.

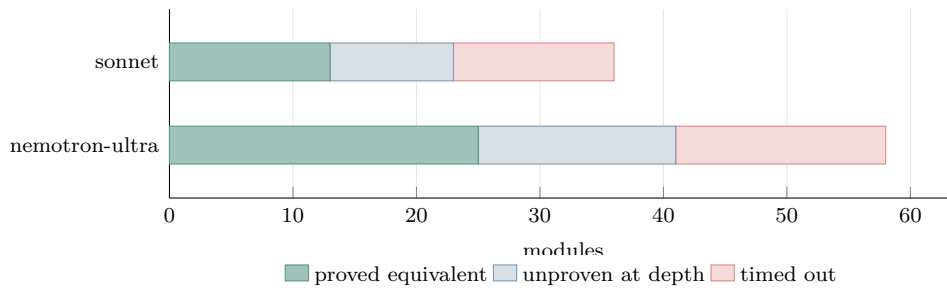


Figure 6: Netlist-against-RTL equivalence per routed variant.

Neither of the two right-hand bands is a counterexample: no module in this run was shown inequivalent. They are the prover running out of depth or time, and they sit on the blocks whose timing is hard for the same reason.

the depth and time budget, not shown inequivalent; no counterexample was found anywhere. They cluster on the deep combinational blocks and the wide AXI interconnect, which is the difficulty this project exists to attack: the blocks a prover struggles with are the blocks whose timing does not close.

11 Deliverable 7 — Interactive demo

`demo_v2.py` walks the flow in eight acts, in the order it actually runs: the sliced critical paths, the four arms over identical targets, the gate funnel, the edits that survived, the clock audit and its runt-pulse control, netlist equivalence, routed PPA, and finally a diff of the report’s own macros against the artifacts the demo just read. It is read-only: no API key, no network, no toolchain. It finishes in about a second. A demo that depends on a free-tier provider being up is a demo that fails in front of an audience.

Act 3 is the part that demonstrates the gate rather than the generator: it shows equivalent-but-wrong rewrites being caught and names the gate that caught each one. For a live run, `bench/scripts/loop.py` takes the same targets file and re-executes the whole loop against a backend of choice; the recorded acts are that command’s output from the runs this report is about.

12 Model comparison

4 models ran in one invocation, over the same three targets, with the same prompt, the same retry budget and the same gate configuration. Only the backend changed, which is what makes the rows comparable. One row is still not readable: an arm that lost calls to the harness defect attempted fewer real edits than its proposal count suggests, so the count is printed next to it rather than averaged away.

Table 12: Four models over the same three targets, one invocation, identical gate configuration. Acceptance means every gate passed, not that the model produced plausible-looking RTL.

Model	Proposals	Accepted	Cost (USD)	Harness losses	Wall (h)
gemini: gemini-flash	3	0	0.01	—	0.1
openrouter: nemotron-ultra	21	1	0.00	0 of 4	3.1
anthropic: sonnet-direct	13	1	2.65	7 of 8	0.5
anthropic: opus-direct	13	0	13.51	8 of 9	0.6

Harness losses are calls this harness threw away rather than the model declining: replies that exhausted the token budget inside reasoning and emitted no text. They are disclosed because an arm that lost calls attempted fewer real edits than its proposal count suggests, and comparing accept rates across arms without that is comparing two different experiments.

12.1 What survived

2 of 4 arms produced a rewrite that passed every gate, and the two that did converged: the same module, the same transform class, found independently. One of them cost \$0.00 (openrouter:nemotron-ultra, on a free endpoint) and took 3.1 h. The other cost \$2.65 (anthropic:sonnet) and took 0.5 h. The difference on this workload is therefore about six times the wall clock against about three dollars. The routed results do not distinguish the two: the netlists agree to within 0.005 ns.

12.2 What the spend did not buy

anthropic:opus was the most expensive arm at \$13.51 and finished with 0 accepted rewrites. We do not read that as a ranking. Of its 13 calls, 9 returned no usable text at all, which is this

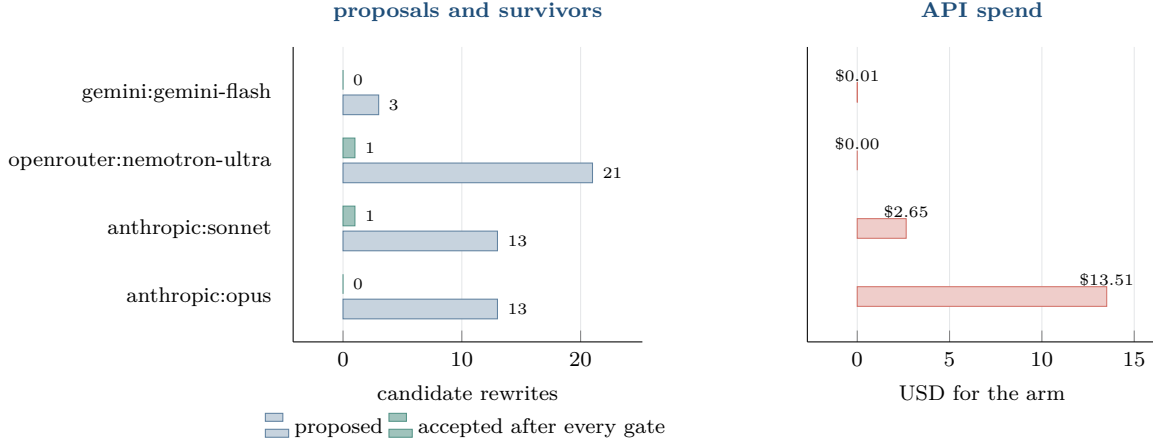


Figure 7: Four models, one invocation, the same three targets and the same gates. Spend and survival are not related the way the price list suggests.

Same run as Table 12. The accepted bar counts rewrites that passed every gate, not replies that looked like Verilog.

harness truncating a reply mid-reasoning rather than the model declining, and the defect was fixed after these runs. This arm spent the most and produced the fewest usable proposals, but the run was not a fair test of it, so no conclusion about the model is drawn from it.

12.3 Where each arm’s candidates died

Table 13: Where each arm’s candidates stopped. Same four arms, same targets, same gate configuration as Table 12.

Model	propose	validate	g1	g1b	accepted	backend
gemini:gemini-flash	—	—	1	—	—	1
openrouter:nemotron-ultra	4	10	4	1	1	—
anthropic:sonnet	8	—	2	1	1	—
anthropic:opus	9	—	2	1	—	—

propose counts calls that returned no usable text, which on three of these arms is the harness defect rather than the model refusing. **validate** counts proposals rejected before any prover ran, for changing an interface or leaving the permitted transform set. **g1** is a failed or unfinished equivalence proof and **g1b** a proof that succeeded at the module and failed at its parent.

Table 13 breaks the losses down by gate. The arms fail at different stages, and the corresponding fixes are different. openrouter:nemotron-ultra reached the gates most often and lost most of its candidates at **validate**, before a prover was ever started, by proposing edits outside the permitted transform set or by changing an interface it was told to leave alone; that is a prompt and parser problem. The two Anthropic arms lost their candidates earlier, inside the call itself, which was the harness defect above. **gemini-flash** ran for minutes, proposed three times, and returned nothing the gates could act on.

One column is the same for every arm that reached it. The three arms that ever cleared **g1** each then lost a candidate at **g1b**, the parent-contract gate: a rewrite proved equivalent to the module it replaced and still wrong in the design. Three models from three vendors produced the same defect, which is the main evidence for checking it mechanically rather than by review.

13 Thought process

Most of the structural decisions in this system came from runs that reported a result we later found to be wrong. Those are the expensive failures here: a crash is caught immediately, while a

flow that reports closure it did not achieve is not. The subsections below are in the order the problems appeared.

13.1 The benchmark had bugs before any model saw it

The first benchmark we wrote passed synthesis, passed simulation, and had two defects in its clock structures: an enable that reached a clock gate without being retimed onto the inactive edge, and a divider output selected by a combinational mux. Both produce runt pulses — a clock edge too narrow to be a clock edge — and neither shows up in a slack number. We found them by writing the audit that is now the G1c/G2c gate, and we fixed them in the benchmark rather than tuning the audit until the benchmark passed. The control in Figure 3 is that audit run against the broken version.

Two rules follow from this and are applied throughout: the input RTL is frozen only after it has been audited, and no gate is used in the flow until it has been shown rejecting a known-bad case.

13.2 The slicer was reading the wrong netlist

Critical paths were originally sliced from the post-resizer netlist, because that is the netlist the timing report comes from. The resizer had already absorbed two of the three violating domains into buffering, so the slicer reported closure the RTL had not achieved and sent the model nothing to do. The slicer now reads the post-synthesis database, where an RTL defect is still an RTL defect.

13.3 Most of the call budget was spent on paths no rewrite can reach

Early runs spent the majority of their API calls on targets where the critical path sits in a macro, a clock network, or an I/O constraint — places none of the four permitted transforms can touch. The model returned a plausible rewrite of the nearest module and the gates rejected it. Targets now carry a lever annotation: no lever, no call, still reported and still counted as a violation. The same budget then buys proposals against paths a rewrite can actually move.

13.4 A proposal that changed nothing was counted as a closed path

One arm re-proposed an already-accepted edit under a second module’s name. The proposal was well-formed and the gates passed, because the tree already contained the edit and there was nothing left to disprove, so the loop counted a second closed path. Scope is now derived from the working tree’s diff instead of from the proposal’s own module list, and a proposal with an empty diff is recorded as a no-op (Section 6.3).

13.5 Pipelining forced the parent into the proof

Two of the four permitted transforms change latency, and no single-module equivalence check can accept a latency change. The first fix was the `submodules` channel: one proposal rewrites the target and every parent that must learn to wait, and the set is proved together. That produced the next failure: rewrites that were provably equivalent at the module and wrong at the parent. G1b was added to catch them, and its rejections are Table 7.

13.6 The biggest loss in the funnel was ours

In the first version of Table 5 the largest single cause of rejection was not a gate but replies that spent their whole token budget inside reasoning and returned no text, which the harness recorded as a model failure. The defect hit the reasoning-heavy arms hardest, which biases the

comparison towards the cheaper models. It is fixed in the runs that followed. It is left in the published funnel with the affected arms marked, rather than removed.

13.7 Global WNS was not a usable metric on this design

Global WNS on a five-master design is set by whichever domain is worst, and here that is the divider path no permitted transform can reach. It also moves further between two placements of the same netlist than under any accepted edit. Reporting was changed to per-domain slack, and the headline claim restricted to a change two independently generated rewrites reproduce. The second routed generation was run for the same reason: a wider floorplan with its own baseline, with no figures mixed between generations.

14 Innovation

Five features of this system are absent from the published work compared against in Section 4.3. Each was added after a run failed without it, and each has a measurement in this report behind it.

1. **Equivalence gate with a recorded proof depth.** An edit enters the optimised RTL only with an EQY result attached, and the depth of that result is stored with it. A combinational rewrite is proved complete in one miter cycle. A feed-forward pipeline insertion is proved complete at latency + 2. Every other case is bounded, and the bound is printed next to the result rather than folded into a pass count. A flow that accepts on simulation does not separate these three cases.
2. **A separate gate for the parent’s timing contract (G1b).** Module-level equivalence is not sufficient, and the gap is measured here rather than argued: 3 rewrites passed G1 with a complete proof and were rejected by G1b at the parent (Table 7). The three rewrites came from three different models, so the defect follows the task rather than one model. This is the report’s main result.
3. **Multi-module proposals, so latency-changing edits are reachable.** Pipelining changes latency, and a single-module equivalence check rejects any latency change. A proposal therefore carries a `submodules` field: it rewrites the target and every parent that must wait for it, and the set is proved together. Without that field the only reachable edits are latency-preserving ones, which rules out the transform that deep combinational paths need.
4. **Accepted scope taken from the working-tree diff.** The loop computes what an edit changed by diffing the tree, not by reading the module list in the proposal. A proposal whose diff is empty is recorded as a no-op rather than a closed path (Section 6.3). Without this, one accepted edit re-sent under a second module’s name is counted twice and the headline number rises with no work behind it.
5. **Slack reported per clock domain, with the noise floor stated.** The design has five mutually asynchronous masters, so a single global WNS mixes five independent questions. Global WNS here is set by a divider path that no permitted transform reaches, and it moves further between two placements of the same netlist than under any accepted edit. The headline claim is therefore restricted to the domain where two structurally different rewrites agree to within 0.005 ns; domains where the two disagree in sign are reported as placement noise.

15 What this does not claim

- The -126.6293 ns divider path did not close. No transform in the permitted set reaches it without a latency change, and the parent handshake that a latency change requires is the

open problem.

- Gains are per-domain. Chip frequency is still governed by the slowest domain, and that domain is the one above.
- Modules that time out in the netlist check are unproven, not proved. We report the distinction rather than the aggregate.
- The clock structures were repaired after the audit found them, and the repair is backed by a runt-pulse simulation (Table 3), not by the structural auditor alone. The auditor matches known unsafe shapes and cannot detect one it has no rule for, so G2c is a regression gate, not a clock-quality certificate.
- CDC gates assume Gray or handshake encoding on multi-bit crossings rather than proving it. The assumption is recorded in every run’s warnings, including the baseline’s.

A The accepted rewrite, in full

Table 6 says an edit to `fp8_adder` was accepted and proved. This is that edit, cut out of `diff -u` at build time rather than transcribed, so it cannot drift from the RTL that was actually measured.

Both models found the same target, a nine-iteration sequential leading-one search written as a `for` loop that synthesises to nine serial compares, and both replaced it with a single-level parallel priority match. The two implementations differ in size by an order of magnitude: one changed 38 lines, the other 299. The routed results still land within 0.005 ns of each other, which is why the improvement is attributed to the transform rather than to either model.

sonnet. 38 changed lines in total; the hunk that carries the transform:

```
@@ -185,17 175,25 @@
         sing_reg = b[7];
     end

-    ////////// leading one detector
-
+    ////////// leading one detector -- rewritten as a balanced priority casez
+    ////////// instead of a 9-iteration sequential for loop. Same priority
+    ////////// order (highest set bit wins), same result every cycle, but
+    ////////// the dependency chain collapses from 9 serial compares to a
+    ////////// single-level parallel priority match.
+
-    for (i = 8; i >= 0; i = i - 1)
-    begin
-        if ((sum_mant[i] == 1'b1) && (found == 1'b0))
-        begin
-            leading_1_index = i[4:0];
-            found = 1'b1;
-        end
-    end
+    casez (sum_mant[8:0])
+        9'b1??????? : leading_1_index = 5'd8;
+        9'b01??????? : leading_1_index = 5'd7;
+        9'b001??????? : leading_1_index = 5'd6;
+        9'b0001??????? : leading_1_index = 5'd5;
+        9'b00001????? : leading_1_index = 5'd4;
+        9'b000001????? : leading_1_index = 5'd3;
+        9'b0000001??? : leading_1_index = 5'd2;
+        9'b00000001?: leading_1_index = 5'd1;
+        9'b000000001: leading_1_index = 5'd0;
+        default:      leading_1_index = 5'd0;
+    endcase
+    found = |sum_mant[8:0];
```

nemotron-ultra. 299 changed lines in total; the hunk that carries the transform:

```

@@ -46,249 46,132 @@
@@ ... 126 lines earlier in the same hunk ...
    mant_a_reg = b_grater_reg ? {2'b00, ({mant_a, 4'd0}) >> shift_b} | {7'b0, sticky_a} : {2'b00, {
mant_a, 4'd0}};
    mant_b_reg = a_grater_reg ? {2'b00, ({mant_b, 4'd0}) >> shift_a} | {7'b0, sticky_b} : {2'b00, {
mant_b, 4'd0}};

    // Complement for subtraction
    a_operand = ((b[6:0] > a[6:0]) && (a[7] ^ b[7])) ? ~mant_a_reg : mant_a_reg;
    b_operand = ((a[6:0] > b[6:0]) && (a[7] ^ b[7])) ? ~mant_b_reg : mant_b_reg;
    sum_mant = a_operand + b_operand + {10'b0, (a[7] ^ b[7])};

-    if (shift_b == 4'd0)
-        sticky_a = 1'b0;
-    else if (shift_b >= 4'd8)
-        sticky_a = |{mant_a,4'd0};
-    else
-        sticky_a = |(({mant_a,4'd0}) & (8'hFF >> (8 - shift_b)));
// Sign
sing_reg = (a[7] == b[7]) ? a[7] : (a[6:0] >= b[6:0] ? a[7] : b[7]);

-    if (shift_a == 4'd0)
-        sticky_b = 1'b0;
-    else if (shift_a >= 4'd8)
-        sticky_b = |{mant_b,4'd0};
-    else
-        sticky_b = |(({mant_b,4'd0}) & (8'hFF >> (8 - shift_a)));
-
-
// Leading-one detector: parallel casez instead of for-loop priority chain
casez (sum_mant[8:0])
    9'b1????????: leading_1_index = 5'd8;
    9'b01????????: leading_1_index = 5'd7;
    9'b001???????: leading_1_index = 5'd6;
    9'b0001?????: leading_1_index = 5'd5;
    9'b00001?????: leading_1_index = 5'd4;
    9'b000001????: leading_1_index = 5'd3;
    9'b0000001???: leading_1_index = 5'd2;

```

sonnet_wide. 183 changed lines in total; the hunk that carries the transform:

```

@@ -185,20 207,27 @@
    sing_reg = b[7];
end

-
-    ////////// leading one detector
-
-    ////////// leading one detector (flat casez priority encoder)
-
-    for (i = 8; i >= 0; i = i - 1)
-    begin
-        if ((sum_mant[i] == 1'b1) && (found == 1'b0))
-        begin
-            leading_1_index = i[4:0];
-            found = 1'b1;
-        end
-    end
-
casez (sum_mant[8:0])
    9'b1????????: begin leading_1_index = 5'd8; found = 1'b1; end
    9'b01????????: begin leading_1_index = 5'd7; found = 1'b1; end
    9'b001???????: begin leading_1_index = 5'd6; found = 1'b1; end
    9'b0001?????: begin leading_1_index = 5'd5; found = 1'b1; end
    9'b00001?????: begin leading_1_index = 5'd4; found = 1'b1; end
    9'b000001????: begin leading_1_index = 5'd3; found = 1'b1; end
    9'b0000001???: begin leading_1_index = 5'd2; found = 1'b1; end
    9'b00000001?: begin leading_1_index = 5'd1; found = 1'b1; end
    9'b000000001: begin leading_1_index = 5'd0; found = 1'b1; end
    default:      begin leading_1_index = 5'd0; found = 1'b0; end
endcase

-    ////////// normalization

```

```

//////// normalization -- candidates for every branch computed
//////// unconditionally in parallel; the branch conditions only pick
//////// among finished values (single flat mux level) instead of the
//////// original 3-deep nested if/else forcing each candidate's

```

B The contract the model is held to

This is the system prompt, quoted out of `bench/scripts/loop.py` at build time. Almost every constraint in it was added because a run failed without it: the reset rule exists because a pipeline stage given a reset the original module did not have disagrees with its own unpipelined self while reset is asserted, and the equivalence checker correctly reports that as a counterexample. The rule against replacing structural arithmetic with a behavioural operator exists because three such answers cost 2400s each and returned no verdict at all. Rewriting a restoring divider as `a / b` does not shorten the path; it hands the prover a hard arithmetic theorem.

The diagnosis fields are read directly into this report, so the prompt requires them to describe the path in front of the model rather than give general timing-closure advice.

You optimise Verilog RTL to close timing. You are given one critical path from a placed and routed design, the RTL module that owns most of its delay, and a catalogue of transforms you may apply.

Reply with a single JSON object and nothing else:

```

{"transform": "<catalogue name>",
 "module": "<module you are rewriting>",
 "latency_delta": "<integer cycles the module's outputs are delayed by>",
 "rtl": "<the complete rewritten module, module...endmodule>",
 "submodules": {"<child module name>": "<its complete rewritten source>"},
 "which_path": "<the failing path in one sentence: what it runs from, what
               it runs to, and which module or instances own the delay>",
 "why": {"cause": "<exactly one of: logic_depth, high_fanout, clock_skew,
               net_delay, congestion, low_drive_strength, other>",
         "evidence": "<the numbers above that support that cause>"},
 "what": {"rtl_level_fix": "<the transform you are applying, and why it
               shortens this path>",
         "tool_level_fix": "<what synthesis or place-and-route could do
               about this path instead, or \"none\" if the
               tools cannot reach it>"}

```

``submodules`` is optional and usually absent. Include it only when the transform genuinely spans two levels -- pipelining a combinational child, for example, puts the stage registers inside the child and the control that tracks them in the parent, and neither half is correct on its own. A child listed there may change its own port list, because parent and child are replaced together. A child that anything OUTSIDE this proposal instantiates may not be listed at all; that is checked and will reject your answer.

Rules that are checked mechanically and will reject your answer:

- ``rtl`` must define exactly the module named in ``module``, with an identical port list: same names, same directions, same widths. This applies to the target module only; a module listed in ``submodules`` may change its ports.
- ``latency_delta`` must be 0 for a latency-preserving transform, and the exact number of cycles added for a latency-changing one. A wrong value is a rejection, not a rounding error.
- You may not edit a module that carries a clock-domain crossing.
- Preserve the module's existing reset behaviour. In particular, a register you ADD to a feed-forward datapath must not be given a reset the original module did not have. Such a register holds no state that has to be recovered, a reset on it is wasted area, and it makes the module disagree with its own unpipelined self for as long as reset is asserted -- which the equivalence checker reports as a counterexample. Write the new stage as a bare ``always @(posedge clk) q <= d;` with no reset branch.
- Do not replace an explicit structural arithmetic block with a behavioural operator. Rewriting a 32-step restoring divider as `a / b`, or an array multiplier as `a * b`, does not shorten the path: synthesis expands the operator back into a comparable structure, so the same logic depth comes back under a different name. It also hands the equivalence checker the task of proving a hand-written division algorithm equal to the ``/``

operator, which is a hard arithmetic theorem and not the question anyone asked -- three such answers cost 2400 seconds each and returned no verdict at all. If an arithmetic block is the bottleneck, pipeline it, retime it or restructure it INTO stages; do not delete its structure.

``which_path``, ``why`` and ``what`` are the diagnosis, not decoration. They are read straight into the report, so write them about THIS path with the numbers from the timing data above, not as general advice about timing closure.

``what.tool_level_fix`` is asked for even though you are not applying it, and answering it honestly matters: most of these paths are already past what the tools can do -- the design has been through resizing and buffering before you see it -- and saying so is the argument for why an RTL edit was needed at all. Write "none" and say why, rather than inventing a tool fix that would not work.

SCOPE. ``module`` is the module you are rewriting and it must be the TARGET module named in the prompt, not one of the modules it instantiates. If the fix needs a child rewritten too, put the child in ``submodules`` and keep the target in ``module``. Rewriting a child alone, with the child's name in ``module``, is rejected without being read: the loop can only splice an edit it asked for.

LATENCY. An edit that adds a pipeline stage changes when the target's outputs arrive, and every parent that consumes them in the same cycle it drives the inputs then breaks. On this design that has rejected four proposals. So a latency change is only worth proposing as ONE proposal that rewrites the target AND, through ``submodules``, every parent that has to learn to wait -- a valid/ready or stall handshake, written out, not assumed. A deep combinational block whose only fix is multi-cycle is exactly this case: it is allowed, it is not a new module, and it is the one way such a path closes.

If no catalogue transform helps this path, reply `{"refused": "<why>"}`.